

CV Creation using LLMs

Abstract

My capstone project develops an automated CV-generation pipeline using two local open-source LLMs. It generates CVs from unstructured candidate data using PDF/WORD input. First, candidate data is processed using the Gemma 3 1B model, which is hosted locally using Ollama. The Gemma 3 1B model converts the data to Candidate information in a structured JSON format and generates ATS-friendly CV content. Second, each CV is reviewed by the Llama 3.2 1B model independently for keyword coverage, fact-checking, and review of formatting. The application is written in Python and comprises a single executable `main.py` file that integrates local models, handles JSON processing, and creates a CV in DOCX format. The application separates the three primary steps and therefore allows the second model to review each CV in an independent manner. The project also includes candidate CV samples, a job description, structured outputs, and evaluation files. The application also includes a fallback mechanism to ensure continuity if Ollama is not available with the primary workflow that utilizes both LLMs.

1. Introduction

Because resumes come in different formats, recruiters have to make comparisons with job descriptions when reviewing resumes. Repetitive activities like this leave recruiters vulnerable to missing critical experience or ATS keywords. With this project, we automate the workflow, treating the candidate document as primary data. The goal is to show how a local LLM can understand, generate, and verify content with a given control and be used practically for document comprehension.

2. Problem Statement

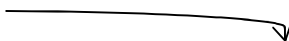
- Project Setup
 - Select an open-source LLM (e.g., Gemma via Ollama or LlamaIndex).
 - Set up a local environment with required dependencies: Python, relevant LLM backends (Ollama, Transformers), and document parsing libraries (e.g., pdfplumber for PDF extraction).
 - Clone and adapt an open-source repository as a starting point, such as ResumeLM.
- Resume Data Extraction
 - Implement a component to convert user CVs from PDF/Word to structured text (or JSON).
 - Use the LLM to extract key resume information: personal details, education, work history, skills, achievements, and projects.

- A typical prompt: “Extract the following fields from this resume: name, contact, education, experience, skills, achievements.”
- Job Description Parsing
 - Accept user-provided job descriptions (copy-pasted or scraped text).
 - Use the LLM to extract requirements, responsibilities, and keywords from the job posting.
 - Store this as a structured object (JSON) for downstream use.
- Resume Tailoring and Generation
 - Summarize or reorganize the candidate’s experience to align with job requirements using prompt engineering.
 - Generate a revised, ATS-friendly CV. For example:
 - Step 1: LLM reviews extracted user info and target job keywords.
 - Step 2: LLM synthesizes or rewrites sections, ensuring personalized and keyword-rich content.
 - Output a ready-to-use DOCX or PDF using template engines and document conversion tools for formatting (e.g., Pandoc).
- User Review and Iterative Revision
 - Allow the user to edit generated content, then use the LLM to refine sections as needed (especially for missing keywords or achievements).
 - Provide real-time feedback on ATS compatibility and keyword optimization.

3. Objectives

Design and develop a capstone project on "CV Creation using LLMs" to showcase how modern open-source models can automate the creation of CV from information provided by user or some documents.

4. Methodology

Workflow: Job description → Gemma JD parsing 

Unstructured profile → Gemma resume extraction → Structured JSON → Gemma CV generation → Llama review → DOCX CV and evaluation JSON.

Gemma 3 1B was chosen because the project outline indicates Gemma 3 1B is a lightweight open-source model which is appropriate for local, privacy-conscious resume tasks. Llama 3.2 1B is used as a separate reviewer to ensure generation and evaluation are not done by the same model. Python, requests, and python-docx are used for automation and document creation.

5. Execution Instructions

- a. Prerequisites
 - i. Python **3.13.2** recommended

- ii. Ollama installed and available in PATH
 - iii. LibreOffice installed and available in PATH for legacy .DOC input
- b. Install Python dependencies
 - i. `python -m pip install --upgrade pip`
 - ii. `python -m pip install -r requirements.txt`
- c. Verify LibreOffice (.DOC support)
 - i. **where soffice**
Expected result should point to something similar to:
C:\Program Files\LibreOffice\program\soffice.exe
If "where soffice" returns nothing, add the LibreOffice "program" directory to PATH and open a new terminal.
- d. Verify Ollama
 - i. **ollama -version**
 - ii. Pull the required local LLM models
 - `ollama pull gemma3:1b`
 - `ollama pull llama3.2:1b`
 - iii. Start Ollama
 - `ollama serve`
 - Keep this terminal running.
- e. Run the application
 - i. `python main.py`

6. Results and Analysis

This addresses the goal of the Capstone_Project-HPPCS[01] to develop final CVs from user-supplied text. It maintains the candidate source as the factual source, calculates deterministic keyword coverage against the parsed job description, and retains the structured JD for auditability. An independent assessment considers overall score, keyword coverage, factual consistency, and format score. This brings together a qualitative LLM review and deterministic safeguards. The produced CVs have standard headings used by ATS and professional content, while unsupported job description skills are not added to the CV as false claims just because they are in the job posting.

7. Conclusion and Future Work

This project builds an end-to-end local LLM CV pipeline with a second-model evaluation stage. It can be extended to include many features such as parsing of job descriptions, automatic generation of CVs for different jobs, editable CVs, feedback for ATS optimization, and a DOCX generator.